

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par : Adam Rekhis

Année Universitaire : 2025-2026

Table des matières

1	Introduction et Contexte du Projet	2
1.1	Contexte métier	2
1.2	Objectifs pédagogiques	2
1.3	Dataset	2
2	Configuration de l'Environnement de Travail	2
2.1	Structure du projet	2
2.2	Environnement virtuel et dépendances	3
3	Exploration et Analyse des Données (EDA)	3
3.1	Description des features	3
3.2	Problèmes de qualité identifiés	4
4	Préparation des Données (preprocessing.py)	4
4.1	Vue d'ensemble du pipeline	4
4.2	Feature Engineering	5
4.3	Correction des valeurs aberrantes	5
4.4	Imputation des valeurs manquantes	5
4.5	Encodage des variables catégorielles	6
4.6	Suppression de la multicollinéarité	6
4.7	Split Train/Test et normalisation	6
5	Modélisation (train_model.py)	7
5.1	Module 1 ACP + Clustering K-Means	7
5.2	Module 2 Classification Churn (Prédiction de départ)	7
5.3	Module 3 Régression (Estimation MonetaryTotal)	8
6	Fonctions Utilitaires (utils.py)	9
7	Prédiction (predict.py)	9
7.1	Chargement des artefacts	9
7.2	Pipeline de prédiction	9
8	Déploiement Application Flask (app/app.py)	10
8.1	Architecture de l'application	10
8.2	Formulaire et réponse JSON	10
8.3	Démarrage de l'application	10
9	Résultats et Discussion	11
9.1	Bilan du pipeline de prétraitement	11
9.2	Recommandations métier	11
10	Conclusion	11

1 Introduction et Contexte du Projet

1.1 Contexte métier

Ce projet s'inscrit dans le cadre d'un atelier de Machine Learning appliqué à l'analyse comportementale de la clientèle d'une boutique e-commerce de cadeaux. En tant que data scientist au sein de cette entreprise, l'objectif est de mieux comprendre la base clients afin de :

- **Personnaliser** les stratégies marketing selon les profils clients.
- **Réduire le taux de churn** (départ des clients).
- **Optimiser le chiffre d'affaires** par une meilleure estimation de la valeur client.

1.2 Objectifs pédagogiques

L'atelier couvre l'intégralité de la chaîne de traitement en data science, de l'exploration des données brutes jusqu'au déploiement d'une interface web :

Compétence	Description
Exploration	Analyser la qualité et la structure des données (52 features, 4 373 clients)
Préparation	Nettoyer, encoder et normaliser les features
Transformation	Réduire la dimension via ACP
Modélisation	Appliquer clustering, classification et régression
Évaluation	Interpréter les résultats et proposer des recommandations
Déploiement	Créer une interface utilisateur avec Flask

1.3 Dataset

Le jeu de données contient **4 373 clients** décrits par **52 features** issues de transactions réelles enrichies de données comportementales et socio-démographiques. Le dataset est intentionnellement imparfait afin de mettre en pratique l'ensemble du pipeline de traitement.

2 Configuration de l'Environnement de Travail

2.1 Structure du projet

L'arborescence respecte les bonnes pratiques de séparation des responsabilités :

```

1 projet_ml_retail/
2 |-- data/
3 |   |-- raw/           # Donnees brutes originales
4 |   |-- processed/     # Donnees nettoyees et encodees
5 |   |-- train_test/    # Splits X_train, X_test, y_train, y_test
6 |-- notebooks/
7 |   |-- exploration.ipynb # Analyse exploratoire (EDA)
8 |-- src/
9 |   |-- utils.py        # Fonctions utilitaires (plots, metriques)
10 |   |-- preprocessing.py # Nettoyage, encodage, normalisation
11 |   |-- train_model.py  # Entrainement clustering, classification, regression
12 |   |-- predict.py      # Prediction sur nouveaux clients
13 |-- models/            # Modeles sauvegardes (.joblib)
14 |-- app/
15 |   |-- app.py          # Application Flask
16 |   |-- templates/
17 |       |-- index.html  # Interface utilisateur
18 |-- reports/           # Graphiques et rapports generes
19 |-- requirements.txt
20 |-- README.md

```

```
21 \-- .gitignore
```

Listing 1 – Structure du projet

2.2 Environnement virtuel et dépendances

Un environnement virtuel Python est créé pour isoler les dépendances du projet :

```
1 # Creation de l'environnement virtuel
2 python -m venv venv
3
4 # Activation (Windows)
5 venv\Scripts\activate
6
7 # Generation du fichier de dependances
8 pip freeze > requirements.txt
9
10 # Reproduction de l'environnement
11 pip install -r requirements.txt
```

Listing 2 – Création et activation de l'environnement virtuel

Fichier README.md

Le fichier README.md documente le projet avec : le titre et la description, les instructions d'installation, la structure expliquée et le guide d'utilisation des scripts.

3 Exploration et Analyse des Données (EDA)

3.1 Description des features

Le dataset comporte 52 features réparties en deux grandes catégories.

Features numériques (34 au total)

TABLE 1 – Features numériques clés

N°	Feature	Type	Description	Intervalle
1	CustomerID	Entier	Identifiant unique client	10000–99999
2	Recency	Entier	Jours depuis dernier achat	0–400 j
3	Frequency	Entier	Nombre de commandes distinctes	1–50
4	MonetaryTotal	Float	Somme totale dépensée	-5000–15000 €
5	MonetaryAvg	Float	Montant moyen par commande	5–500 €
6	CustomerTenure	Entier	Durée relation client	0–730 j
7	Age	Float	Âge estimé (30% manquant)	18–81 ou NaN
8	SupportTickets	Float	Tickets support (-1, 0–15, 999)	Aberrants
9	Satisfaction	Float	Note satisfaction (-1, 0, 1–5, 99)	Aberrants
10	Churn	Binaire	Indicateur départ client	0 / 1

Features catégorielles (18 au total)

TABLE 2 – Features catégorielles

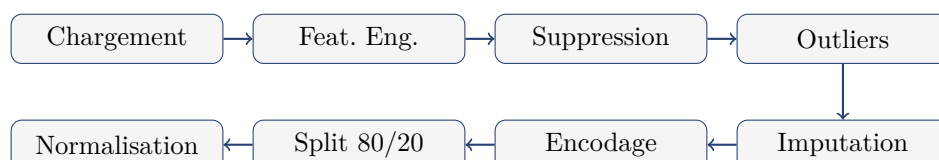
N°	Feature	Card.	Valeurs	Encodage
35	RFMSegment	4	Champions, Fidèles, Potentiels, Dormants	Ordinal
36	AgeCategory	7	18-24, 25-34, ..., 65+, Inconnu	Ordinal
37	SpendingCat	4	Low, Medium, High, VIP	Ordinal
38	CustomerType	5	Hyperactif, Régulier, Occasionnel, Nouveau, Perdu	One-Hot
39	FavoriteSeason	4	Hiver, Printemps, Été, Automne	One-Hot
40	PreferredTime	5	Matin, Midi, Après-midi, Soir, Nuit	One-Hot
41	Region	8	UK, Europe (N/S/E/C), Asie, Autre	One-Hot
42	LoyaltyLevel	5	Nouveau, Jeune, Établi, Ancien, Inconnu	Ordinal
43	ChurnRisk	4	Faible, Moyen, Élevé, Critique	Ordinal
49	Country	37+	UK, France, Germany, etc.	Target Enc.
50	Newsletter	1	Yes (constante)	Supprimée
51	RegistrationDate	Var.	Formats inconsistants	Parsing
52	LastLoginIP	Unique	Adresse IP	Feature Eng.

3.2 Problèmes de qualité identifiés

Type	Features	Fréq.	Traitement
Valeurs manquantes	Age	30%	KNN Imputer (k=5)
Valeurs aberrantes	SupportTickets, Satisfaction	5-8%	Remplacement par NaN
Formats incohérents	RegistrationDate	100%	Parsing to_datetime
Feature constante	NewsletterSubscribed	100%	Suppression
Données brutes	LastLoginIP	100%	Extraction IsPrivateIP
Déséquilibre classes	Churn, AccountStatus	Var.	SMOTE + class_weight

4 Préparation des Données (preprocessing.py)**4.1 Vue d'ensemble du pipeline**

Le script `preprocessing.py` implémente un pipeline séquentiel complet transformant les données brutes en ensembles d'entraînement/test prêts pour la modélisation.



4.2 Feature Engineering

De nouvelles features sont créées pour mieux capturer le comportement client :

```

1 # Ratio dépenses / recency
2 df['MonetaryPerDay'] = df['MonetaryTotal'] / (df['Recency'] + 1)
3
4 # Valeur moyenne du panier
5 df['AvgBasketValue'] = df['MonetaryTotal'] / (df['Frequency'] + 1)
6
7 # Rapport anciennete / activite recente
8 df['TenureRatio'] = df['Recency'] / (df['CustomerTenureDays'] + 1)
9
10 # Taux d'annulation
11 df['CancelRate'] = df['CancelledTransactions'] / (df['TotalTransactions'] + 1)
12
13 # Parsing RegistrationDate (formats inconsistants : UK, ISO, US)
14 reg = pd.to_datetime(df['RegistrationDate'], dayfirst=True, errors='coerce')
15 df['RegYear'] = reg.dt.year
16 df['RegMonth'] = reg.dt.month
17 df['RegWeekday'] = reg.dt.weekday
18
19 # Extraction depuis LastLoginIP
20 df['IsPrivateIP'] = df['LastLoginIP'].apply(is_private_ip) # 0 ou 1

```

Listing 3 – Nouvelles features métier créées

4.3 Correction des valeurs aberrantes

```

1 # SupportTicketsCount : -1 et 999 sont des codes erreurs
2 df['SupportTicketsCount'] = df['SupportTicketsCount'].replace(
3     {-1: np.nan, 999: np.nan})
4
5 # SatisfactionScore : seules les valeurs entre 0 et 5 sont valides
6 df['SatisfactionScore'] = df['SatisfactionScore'].where(
7     df['SatisfactionScore'].between(0, 5), np.nan
8 )
9 # Winsorisation IQR (facteur 3) pour les autres numeriques
10 def cap_outliers_iqr(df, cols, factor=3.0):
11     for col in cols:
12         q1, q3 = df[col].quantile(0.25), df[col].quantile(0.75)
13         iqr = q3 - q1
14         df[col] = df[col].clip(q1 - factor*iqr, q3 + factor*iqr)

```

Listing 4 – Détection et correction des outliers

4.4 Imputation des valeurs manquantes

```

1 # KNN Imputer specifique pour Age (30% de valeurs manquantes)
2 ref_cols = ['Age', 'Recency', 'Frequency', 'MonetaryTotal']
3 imp = KNNImputer(n_neighbors=5)
4 df[ref_cols] = imp.fit_transform(df[ref_cols])
5 df['Age'] = df['Age'].clip(18, 81) # Contrainte metier
6
7 # Imputation mediane pour les autres colonnes numeriques
8 imp_median = SimpleImputer(strategy='median')
9 df[num_cols] = imp_median.fit_transform(df[num_cols])

```

Listing 5 – Imputation KNN pour Age (30% manquant)

Règle importante : éviter le Data Leakage

Le `StandardScaler` et les imputeurs sont **fit** uniquement sur `X_train`, puis **transform** seulement sur `X_test`. Appliquer le `fit_transform` sur l'ensemble complet avant le split constituerait une fuite de données (*data leakage*).

4.5 Encodage des variables catégorielles

Trois stratégies d'encodage sont utilisées selon la nature de chaque variable :

Méthode	Variables concernées	Justification
Encodage ordinal	RFMSegment, SpendingCat, LoyaltyLevel, ChurnRisk, AgeCategory, BasketSize, PreferredTime	Ordre naturel entre catégories
One-Hot Encoding	CustomerType, FavoriteSeason, Region, Gender, AccountStatus, WeekendPreference, ProductDiversity	Catégories nominales sans ordre
Target Encoding	Country (37+ pays)	Haute cardinalité, évite l'explosion dimensionnelle

4.6 Suppression de la multicollinéarité

Une analyse de la matrice de corrélation (seuil $|r| \geq 0.8$) conduit à supprimer les colonnes redondantes :

```

1 HIGH_CORR_DROPS = [
2     'MonetaryMin', 'MonetaryMax',           # MonetaryTotal plus synthétique
3     'MinQuantity', 'MaxQuantity',           # TotalQuantity synthétise tout
4     'AvgQuantityPerTransaction',
5     'UniqueInvoices',                       # Redondant avec TotalTransactions
6     'UniqueDescriptions',                   # Très corrélé à UniqueProducts
7     'FirstPurchaseDaysAgo',                 # Proche de CustomerTenureDays
8 ]

```

Listing 6 – Colonnes supprimées pour réduire la multicollinéarité

4.7 Split Train/Test et normalisation

```

1 # Split stratifié sur Churn (conserver les proportions de classes)
2 X_train, X_test, y_train, y_test = train_test_split(
3     X, y, test_size=0.2, random_state=42, stratify=y
4 )
5
6 # Normalisation : fit sur train uniquement
7 scaler = StandardScaler()
8 X_train_scaled = scaler.fit_transform(X_train)
9 X_test_scaled = scaler.transform(X_test) # pas de fit sur le test !
10
11 # Sauvegarde des splits et du scaler
12 X_train_scaled.to_csv('data/train_test/X_train.csv', index=False)
13 joblib.dump(scaler, 'models/scaler.joblib')

```

Listing 7 – Split stratifié 80/20 et normalisation

5 Modélisation (train_model.py)

Le fichier `train_model.py` regroupe trois modules de modélisation indépendants.

5.1 Module 1 ACP + Clustering K-Means

Analyse en Composantes Principales (ACP)

L'ACP réduit la dimensionnalité tout en conservant l'essentiel de l'information :

```
1 # Reduction de dimension : conservation de 95% de la variance
2 pca = PCA(n_components=0.95, random_state=42)
3 X_pca_train = pca.fit_transform(X_train)
4 X_pca_test = pca.transform(X_test)
5 # Resultat typique : ~52 features -> ~15-20 composantes
```

Listing 8 – ACP avec 95% de variance conservée

L'ACP présente plusieurs avantages dans ce contexte :

- Réduction du bruit et de la dimensionnalité.
- Accélération des calculs de clustering.
- Visualisation possible en 2D/3D.
- Lutte contre la malédiction de la dimensionnalité.

Clustering K-Means

La méthode Elbow combinée au score Silhouette permet de sélectionner le nombre optimal de clusters k :

```
1 inertias, silhouettes = [], []
2 for k in range(2, 9):
3     km = KMeans(n_clusters=k, random_state=42, n_init=10)
4     labels = km.fit_predict(X_pca_train)
5     inertias.append(km.inertia_)
6     sil = silhouette_score(X_pca_train, labels,
7                           sample_size=1000, random_state=42)
8     silhouettes.append(sil)
9
10 # Meilleur k = argmax(silhouettes)
11 best_k = k_range[np.argmax(silhouettes)]
12 kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=20)
```

Listing 9 – Recherche du meilleur k (Elbow + Silhouette)

Les 4 clusters obtenus correspondent à des profils clients distincts :

Cluster	Profil	Caractéristiques
0	Dormants	Recency élevée, faible fréquence, faible montant
1	Occasionnels	Activité sporadique, panier moyen
2	Réguliers	Achat fréquent, ancienneté établie
3	Champions	Recency faible, fréquence élevée, montant élevé

5.2 Module 2 Classification Churn (Prédiction de départ)

Gestion du déséquilibre de classes

Le taux de churn naturellement déséquilibré est corrigé via SMOTE :


```

1 smote = SMOTE(random_state=42, k_neighbors=5)
2 X_res, y_res = smote.fit_resample(X_train, y_train)
3 # Avant SMOTE : {0: ~3500, 1: ~870}
4 # Après SMOTE : {0: ~3500, 1: ~3500}

```

Listing 10 – SMOTE pour le rééquilibrage

Modèles entraînés

Deux classifieurs sont comparés :

Paramètre	Logistic Regression	Random Forest
Rôle	Modèle baseline	Modèle principal
n_estimators	—	300 arbres
max_depth	—	12
class_weight	balanced	balanced
Solver	lbfgs	—
Avantage	Interprétabilité	Performance, robustesse

```

1 rf_params = {
2     'n_estimators': 300,
3     'max_depth': 12,
4     'min_samples_split': 5,
5     'min_samples_leaf': 2,
6     'max_features': 'sqrt',
7     'class_weight': 'balanced',
8     'random_state': 42,
9     'n_jobs': -1, # parallelisation sur tous les coeurs
10 }
11 rf = RandomForestClassifier(**rf_params)
12 rf.fit(X_res, y_res)

```

Listing 11 – Configuration du Random Forest Classifier

Métriques d'évaluation

Pour un problème de classification déséquilibrée, les métriques prioritaires sont : l'**AUC-ROC**, la **précision**, le **rappel** et le **F1-score** pour chaque classe. La matrice de confusion et la courbe ROC sont générées et sauvegardées dans `reports/`.

5.3 Module 3 Régression (Estimation MonetaryTotal)

```

1 # On retire les features monetaires pour eviter la fuite triviale
2 for drop_col in ['MonetaryTotal', 'MonetaryAvg', 'MonetaryStd',
3                 'MonetaryPerDay', 'AvgBasketValue']:
4     X_train_reg = X_train_reg.drop(columns=[drop_col], errors='ignore')
5
6 rfr = RandomForestRegressor(
7     n_estimators=200, max_depth=10,
8     min_samples_split=5, min_samples_leaf=2,
9     max_features='sqrt', random_state=42, n_jobs=-1
10 )
11 rfr.fit(X_train_reg, y_reg_train)

```

Listing 12 – Random Forest Regressor pour MonetaryTotal

Métriques de régression utilisées

- **RMSE** (Root Mean Square Error) : sensible aux grandes erreurs.
- **R^2** (coefficient de détermination) : proportion de variance expliquée par le modèle. Un $R^2 = 1$ indique un ajustement parfait.

6 Fonctions Utilitaires (utils.py)

Le fichier `utils.py` centralise toutes les fonctions de visualisation et d'évaluation. Toutes les figures sont automatiquement sauvegardées dans le répertoire `reports/`.

Fonction	Description
<code>plot_missing_values</code>	Bar chart des pourcentages de valeurs manquantes par feature
<code>plot_distributions</code>	Histogrammes des features numériques (grille $n \times 4$)
<code>plot_correlation_heatmap</code>	Heatmap de corrélation avec détection des paires $ r \geq 0.8$
<code>plot_class_balance</code>	Distribution des classes (Churn, AccountStatus, etc.)
<code>plot_boxplots</code>	Boxplots pour la détection visuelle des outliers
<code>evaluate_classifier</code>	Rapport complet : classification_report, AUC, ROC, confusion
<code>evaluate_regressor</code>	RMSE, R^2 , scatter réel vs prédit, distribution résidus
<code>plot_feature_importance</code>	Bar chart horizontal des importances (modèles arborescents)
<code>plot_pca_variance</code>	Courbe de variance expliquée cumulée (seuils 90% et 95%)
<code>plot_pca_2d</code>	Scatter 2D coloré par cluster après ACP
<code>plot_elbow_silhouette</code>	Courbes Elbow (inertie) et Silhouette pour choix de k

7 Prédiction (predict.py)

7.1 Chargement des artefacts

Le script `predict.py` charge l'ensemble des modèles sauvegardés :

```

1 artefacts = {
2     'scaler':      joblib.load('models/scaler.joblib'),
3     'pca':         joblib.load('models/pca.joblib'),
4     'kmeans':      joblib.load('models/kmeans.joblib'),
5     'classifier':  joblib.load('models/best_classifier.joblib'),
6     'regressor':   joblib.load('models/random_forest_regressor.joblib'),
7 }

```

Listing 13 – Chargement des modèles sauvegardés

7.2 Pipeline de prédiction

Pour chaque nouveau client, la fonction `predict_all` :

1. Aligne les colonnes sur celles vues à l'entraînement (colonnes manquantes $\rightarrow 0$).
2. Applique le `StandardScaler`.

3. Prédit le **cluster** (ACP + K-Means).
4. Prédit la **probabilité de churn** et le label binaire.
5. Estime le **montant monétaire** (après suppression des features monétaires).

```

1 # Prediction sur un exemple interne
2 python src/predict.py
3
4 # Prediction batch depuis un fichier CSV
5 python src/predict.py --file path/to/nouveaux_clients.csv

```

Listing 14 – Utilisation en ligne de commande

8 Déploiement Application Flask (app/app.py)

8.1 Architecture de l'application

Flask est un micro-framework Python léger et flexible, idéal pour servir des modèles ML via une API REST ou une interface web. L'application expose trois routes principales :

Route	Méthode	Description
/	GET	Affiche le formulaire de saisie client (<code>index.html</code>)
/predict	POST	Reçoit les données, applique les modèles, retourne JSON
/health	GET	Statut de l'API et état de chargement des modèles

8.2 Formulaire et réponse JSON

L'interface permet la saisie de 15 features clés. La réponse JSON retournée contient :

```

1 {
2   "success": true,
3   "cluster": 3,
4   "cluster_label": "Champions",
5   "churn_probability": 12.5,
6   "churn_prediction": 0,
7   "churn_label": "Client fidele",
8   "risk_level": "Faible",           // Faible / Modere / Eleve / Critique
9   "predicted_monetary": 2345.67
10 }

```

Listing 15 – Exemple de réponse JSON de l'API

8.3 Démarrage de l'application

```

1 # Depuis la racine du projet
2 python app/app.py
3
4 # Acces via navigateur
5 # http://127.0.0.1:5000          -> Interface utilisateur
6 # http://127.0.0.1:5000/health  -> Statut des modeles

```

Listing 16 – Lancement du serveur Flask

Alternatives à Flask

- **FastAPI** : plus rapide, support asynchrone natif, génération automatique de docs OpenAPI.
- **Django** : plus complet mais plus lourd, adapté aux grandes applications.
- **Streamlit** : solution très rapide pour créer des data apps sans HTML/CSS.

9 Résultats et Discussion

9.1 Bilan du pipeline de prétraitement

La chaîne de prétraitement transforme les 52 features brutes en un ensemble propre et encodé. Les principales transformations appliquées sont résumées ci-dessous :

Étape	Résultat
Suppression inutiles	CustomerID, Newsletter, LastLoginIP (brut), RegistrationDate (brut)
Feature Engineering	8 nouvelles features créées (MonetaryPerDay, AvgBasketValue, TenureRatio, CancelRate, RegYear, RegMonth, RegWeekday, IsPrivateIP)
Correction outliers	SupportTickets et Satisfaction nettoyés ; winsorisation IQR (facteur 3)
Imputation	Age : KNN (k=5) ; autres numériques : médiane
Encodage	7 ordinales encodées, 7 nominales en One-Hot, Country en Target Enc.
Multicolinéarité	7 colonnes redondantes supprimées
Split / Normalisation	80% train / 20% test, StandardScaler fit sur train uniquement

9.2 Recommandations métier

Sur la base des résultats de modélisation, les recommandations suivantes peuvent être formulées :

1. **Clustering** : Cibler les clients *Dormants* avec des campagnes de réactivation (promotions, relances e-mail). Fidéliser les *Champions* avec des programmes de récompense.
2. **Churn** : Surveiller les clients classés *Risque élevé* ou *Critique* par le modèle. Agir proactivement avant que leur départ soit effectif (offres personnalisées, suivi satisfaction).
3. **Valeur monétaire** : Utiliser les prédictions de **MonetaryTotal** pour prioriser les actions marketing vers les clients à fort potentiel de dépense.

10 Conclusion

Ce projet a permis de mettre en uvre l'intégralité de la chaîne de traitement en data science, de l'exploration des données brutes jusqu'au déploiement d'une API Flask opérationnelle.

Les principales compétences exercées sont :

- La gestion de données imparfaites (valeurs manquantes, aberrantes, formats incohérents).
- L'importance du **respect de l'ordre train/test** pour éviter le data leakage.
- La sélection des métriques adaptées selon le type de problème (AUC pour classification déséquilibrée, RMSE/R $\hat{s}$ pour la régression).
- L'intérêt de l'ACP comme étape préalable au clustering pour améliorer la séparation des groupes.
- Le déploiement d'un modèle ML sous forme d'API REST avec Flask.

Les pistes d'amélioration envisageables incluent l'utilisation d'Optuna pour l'optimisation automatique des hyperparamètres, l'ajout d'un module d'explicabilité (SHAP, LIME), ainsi que le passage à une architecture Docker pour la conteneurisation du déploiement.